Building with AI, the way your project decided

project-kit (pkit)

Aleš Kalfas

github.com/aleskalfas/project-kit

Version pkit 1.149 · 30 October 2026

Part 1

pkit at a glance

Why pkit exists, how to install it, and how its parts fit together.

Why pkit?

Your AI agent works fast, and forgets everything your project decided.

Choices stay in chats and in people's heads so the agent cannot know them, and decides again	Capture each decision, with why you accept it, and every session can read it · Part 2
Conventions stay unwritten so the agent falls back on its own habits	Write your conventions down every session loads them, and agents follow them · Parts 2, 5
Docs aren't tied to the code so a code change leaves them false, and nobody notices	Tie each doc to what it depends on each change rechecks the docs it affects · Part 4
What the agent may do is never said so the tool guesses, action by action	Write down what it may do decided once by your project, the same in every session · Part 3

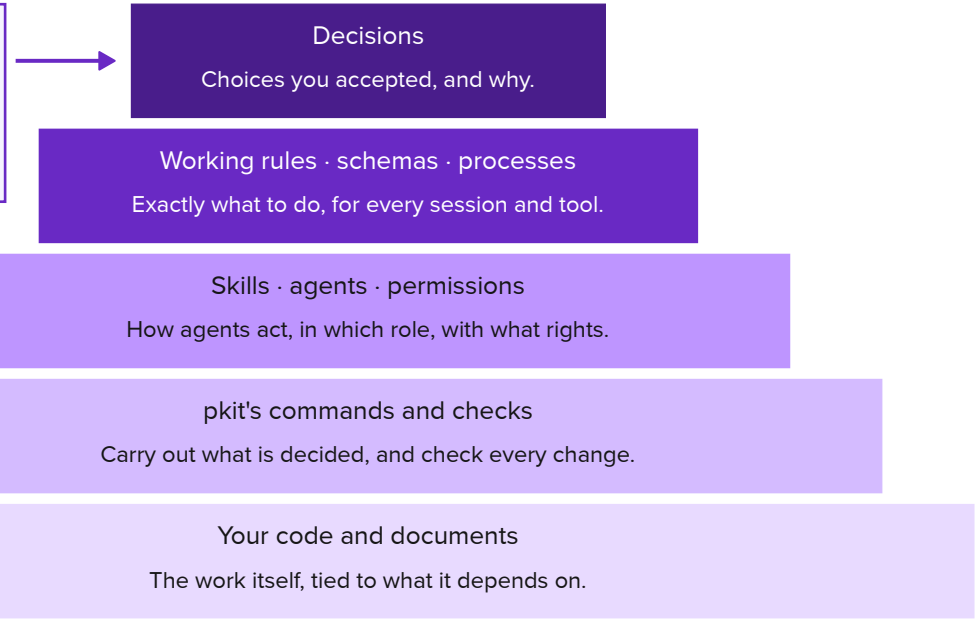
The question: how do you make it work your project's way, without a heavy framework?

How pkit fits together

In five layers, from your decisions down to your code: each carries out the ones above it.

Where ideas start

In conversation, and in a scratchpad note when one needs longer.



Carried out

Each layer carries out the decisions above it. The reasons stay in one place, where an agent reads them before it changes anything.

Checked

If a layer and a decision disagree, the layer is checked first, so decisions never drift to match the code. Only you change a decision.

This week: pkit wrote this order down as a decision of its own, COR-065

Install pkit

Install the tool once, then add pkit to each project with one command.
Upgrades replace pkit's files, never yours.

```
# once, on your machine
$ uv tool install \
    git+ssh://git@github.com/aleskalfas/project-kit.git@latest
$ pkit --version

# in each project
$ cd your-project
$ pkit init
```



Don't want pkit in your repository?

```
$ pkit visibility private
```

pkit stays only in your clone, hidden by git's exclude file. Nothing of it is committed, and your agent still uses all of it.

```
$ tree -a -L 2 your-project
your-project/
├── .pkit/                          # the methodology
│   ├── adapters/                   # the bridge to your AI tool
│   ├── agents/ core/ project/
│   ├── cli/                        # the command's reference
│   ├── decisions/ core/ project/
│   ├── lifecycle/                  # install, sync, upgrade
│   ├── migrations/                 # move your project to new versions
│   ├── permissions/ project/       # what agents may do
│   ├── process/                    # staged processes, as data
│   ├── rules/ core.md project.md   # yours to add
│   ├── schemas/                    # what the tools read
│   ├── scratchpad/                 # your unfinished ideas
│   ├── skills/ core/ project/
│   ├── manifest.yaml               # what is installed, which version
│   └── .gitignore                  # pkit's runtime files
├── .claude/                        # your Claude Code setup
│   ├── agents/  skills/            # pkit's beside yours
│   └── settings.json               # yours, pkit's merged in
├── .agent-workspace/               # agents' scratch, not committed
├── CLAUDE.md                       # loads the rules each session
└── src/                            # your code, untouched
# purple: pkit's, replaced by each upgrade · blue: yours
```

pkit status shows what pkit set up in a project. Optional parts, called capabilities, come later.

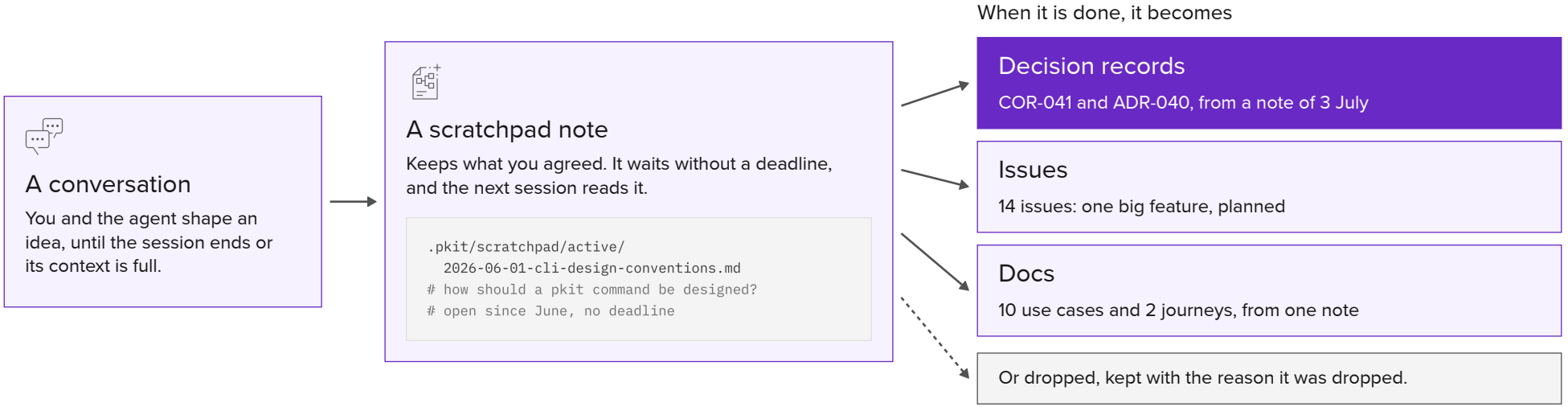
Part 2

From idea to rule

Where ideas start, your decisions, what every session is told, and rules a program applies.

Where ideas start

In conversation. A scratchpad note keeps what you agreed past the session, until it becomes records, issues or docs.



Limit: Nothing writes the note for you: what stays only in the conversation is lost with it.

In project-kit: 28 notes done, which became 41 records, 31 issues and 15 docs

Decision records

A decision record keeps one choice and why it was made. Where it lives says what it is about, and who may change it.

How work is done

`.pkit/decisions/`

What you build, at a path you choose

`docs/architecture/decisions/`

COR

pkit's principles

How the method works, the same in every project.

core/COR-065-records-and-projection.md

Kept apart from yours: each upgrade replaces them, so you never edit them.

PRJ

Your way of working

How your team works, in the choices pkit leaves to you.

project/PRJ-002-version-bump-policy.md

Yours alone: no upgrade ever writes here.

ADR

Your architecture

How your system is built, its structure, technologies and boundaries.

ADR-001-project-root-resolution.md

Kept with your system's docs, where its architect keeps them.

> Limit: Writing and reviewing a record takes time: worth it only for choices that later work builds on.

Decided once, found later

You accept a decision before work builds on it. Later, an agent finds the decision before changing the files it covers.

The agent drafts it

It writes the record, and ties the release guide to it.

```
$ cat .pkit/decisions/project/PRJ-002-….md
id: PRJ-002
title: Version-bump policy
status: proposed    # until you accept it
## Context          # why it came up
## Decision         # what is decided
## Rationale        # why, what was rejected
## Implications     # what follows

# .pkit/release/README.md, its front matter:
  friction:
    anchors:
+     record: [PRJ-002]
```

You accept it

You see it before it merges, while it is cheapest to change.

```
# the pull request, before it merges:
  .pkit/decisions/project/PRJ-002-….md
- status: proposed
+ status: accepted

  .pkit/release/README.md
    anchors:
+     record: [PRJ-002]
```

A later session finds it

It starts with none of your reasons, so it looks up the records first.

```
$ pkit refs records-for .pkit/release/README.md
6 records tied directly:
  PRJ-002  Version-bump policy …
  PRJ-004  Distribution via direct git URL …
  ADR-040  Fetch externally-sourced content …
  COR-010  Lifecycle of installed resources
  …
# the agent reads these before changing it
```

Limit: Only records tied to a file are found, directly or through a document that depends on it.

This week: ADR-075 decided how the rules every session loads are made. It was accepted and merged with its code.

What every session is told

A few short rules on how to work, pkit's and yours. Records are too many to load, so each rule names the one behind it.

The decision (COR-065)

An agent reads the records only when it changes something, not in every session.

The rule every session gets (rule 22)

Before you change files, read the records behind them.

What the agent does

Before it changes a file, it runs pkit refs records-for and reads what it lists.

```
# loaded at the start of every session
.pkit/rules/core.md      # pkit's, in every project
.pkit/rules/project.md   # yours, never touched by pkit

# looked up when needed
$ pkit refs records-for <files>  # the records behind
                                 # what it changes
$ pkit rule-sets reach <paths>   # rules about what
                                 # it writes
```

> Limit: A rule is advice: an agent can still break it.

This week: each rule got a permanent id and a status. The file every session loads is now generated from the rules.

What schemas and processes look like

A schema is two files: the rule as data, and the shape that data must have. A process is a schema of stages and moves.

A schema: two files

In core pkit

The rule, as data

```
# .pkit/schemas/privilege-catalog.yaml
vcs-history-rewrite:
  description: Rewriting published git history (force-push).
  guardrail: true
  recognize:
    bash:
      - cmd: git
        subcommand: [push]
        flag_any: ["--force", "-f", …]
```

The shape it must have

```
# .pkit/schemas/privilege-catalog.schema.json
"entry": {
  "type": "object",
  "additionalProperties": false,
  "required": ["description", "recognize"],
  "properties": {
    "description": {
      "type": "string",
      "minLength": 1,
```

A process: one kind of schema

A capability's example: not in core pkit

The issue workflow

```
# .pkit/capabilities/project-management/
#   schemas/workflow.yaml
process:
  id: issue-lifecycle
  states:  # todo … done, five
  transitions:
    - from: review
      to: done
      authorisation: user
```

What core pkit brings, so no capability and no project builds it again

```
# for schemas
$ pkit new schema <capability> <name>   # a rule and its shape, as a pair
$ pkit validate                         # every file against its shape
$ pkit data validate <file>             # your own data files too
```

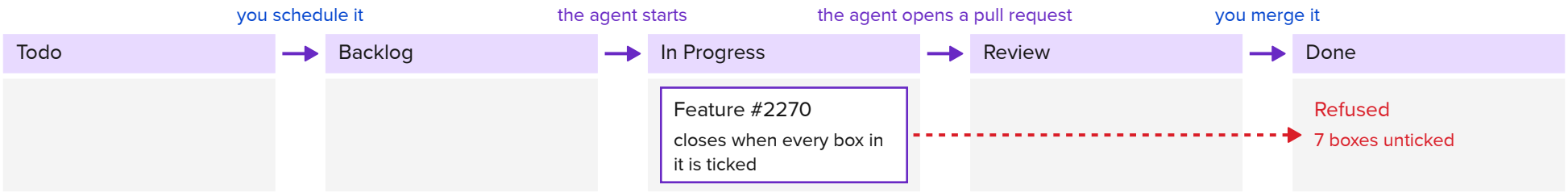
```
# for processes
$ pkit process new <capability>:<id>    # a process, its gates stubbed
$ pkit process status                   # where it is, and why
$ pkit process move                     # the move, refused if not allowed
```

> Limit: pkit validate checks that a file has its shape, not that the rule in it is right.

In a fresh install: 8 schemas of pkit's own, and the shape every process must have. pkit validate checks them all.

How a capability uses a process

project-management brings the issue workflow. pkit's engine checks every move, so the capability wrote no engine of its own.



The move, in the capability's workflow.yaml

```
# workflow.yaml, from project-management
- from: in-progress
  to: done
  authorisation: user
  gate:
    kind: deterministic
    predicate:
      run: gate-checkboxes-ticked
```

Its shape comes from core pkit, the same for every process.

What the agent asks pkit

```
$ pkit process status \
    project-management:issue-lifecycle
    --subject 2270
Where: in-progress — Branch open, work in flight.
Legal moves (live precheck):
  ✗ done  [close-issue]
      7 unticked checkbox(es) remain
```

Why it helps

The agent asks where work stands, instead of guessing.

A move needs its check or your yes, never just the agent's word.

The capability records each move through pkit's engine, and built none of its own.

> Limit: pkit brings the engine, not a process: the one in use today comes with the project-management capability.

Part 3

How the agent works

Procedures, roles, and what each agent may do.

Where skills come from

Each kind of file pkit's decisions define gets a skill for the choices, and a command for the exact part.

The decision defines a kind of file	→	The skill writes one as decided	→	The command creates the file	→	What you get a new file, ready to fill in
COR-012 Scratchpad notes		scratchpad-author		`pkit new scratchpad`		A dated note in active/
COR-006 Decision records		decision-author		`pkit new decision`		The next number, proposed until you accept it
COR-013 Agents		agent-author		`pkit new agent`		An agent file in pkit's shape

Why pkit splits the work this way

The reason lives in one place. The skill points to it and never copies it.	A command cannot pick a name or write the body. Without a skill, agents skipped the writing rules.	An agent can miscount. A record written by hand once got a wrong number.	Every file of a kind has the same shape, whoever writes it.

> Limit: pkit's skills cover only pkit's own files. For work that keeps coming back in your project, you write your own.

In project-kit: pkit ships 11 skills, each with its command.

One skill, step by step

The agent follows the steps in order. At step 6 it runs the command, so the number and the layout are always right.

The skill decision-author: a new decision record

1	Check that its decisions are accepted	
2	Pick the kind of record	
3	Read the writing rules	
4	Read the record's shape	
5	Pick a short name	
6	Create the file	its command
7	Write the body	
8	Check it against the rules	
9	Show you the draft	
10	Commit, after you approve	

Its command: pkit new decision

```
$ pkit new decision project test-coverage-floor
Stamped: .pkit/decisions/project/PRJ-012-test-coverage-floor.md
# in the file:
id: PRJ-012          # the next number, never guessed
status: proposed     # until you accept it
## Context
## Decision
## Rationale
## Implications
```

> Limit: Nothing checks that an agent used the skill: the command creates the file either way.

Every session holds only each skill's short description. The steps load when the task comes up.

Where agents come from

When the same kind of check keeps coming back, a decision names a role for it: one job, and only the tools it needs.

The decision names a role	The agent does one job	Its limit only the tools it needs	What it catches so you do not have to
COR-024 A second opinion	critic	Reads only	A draft's gaps, before you read it
COR-024 A keeper of the whole system	architect	Edits only architecture docs	A change that breaks the whole
COR-057 One reviewer for the rules	rule-set-reviewer	Never edits what it checks	A broken rule, with its line

Why pkit has agents

A decision makes each check one role's job. By default, nobody watches the whole system.

The agent that writes agrees with itself. A second agent does not.

A reviewer that cannot edit stays independent.

Caught here, a mistake costs an agent's time, not yours.

> Limit: The AI tool enforces an agent's tools. Which files it may change is a rule it follows, not a lock.

In project-kit: pkit ships 6 agents, each a file with its job, its tools and the decisions it follows.

One piece of work, and who sees it

Each role sees the work at its own moment: the critic before you, the reviewers on the change.



Limit: Every review round costs several agent runs.

What every agent comes with

Each agent arrives tied to its decisions and pointed at your folders. pkit fills in your paths and checks every tie.

pkit's agent

```
# .pkit/agents/core/architect.md
gates: [COR-024, COR-025, COR-014]
owns:
  - <architecture-docs>
  - <adr-records>
```

+

Your paths

```
# .pkit/agents/project/overlay.yaml
architecture-docs:
  - README.md
adr-records:
  - docs/architecture/decisions/
```

→

The agent your AI tool gets

```
# .claude/agents/architect.md
gates: [COR-024, COR-025, COR-014]
owns:
  - README.md
  - docs/architecture/decisions/
```

Its decisions

Its first lines name the decisions it follows and the documents it reads.

Checked

pkit validate checks every name, and that no two agents own the same file.

Kept current

Each upgrade deploys it again, with your paths.

Do you have to wire it?

```
$ pkit sync   # after you change overlay.yaml
```

No. A fresh install fills in default paths, and all six agents are ready at once.

Limit: You never edit pkit's agents. For a different role, you write your own in .pkit/agents/project/.

In a fresh install: all 6 agents deployed at once, with default paths. pkit validate checked 17 skills and agents.

Where permissions come from

From rules your project writes down as data. Your AI tool applies them to every call, so they hold when an agent forgets them.

The rule decided by COR-028	→ Written as data in files you can read	→ Your AI tool gets and applies itself	→ What a call meets every time
Guardrails for you and every agent	rm -r, sudo and force-push, denied to all	A deny in its settings	Refused, in every mode
An agent's own tools in the agent's file	critic: Read, Glob, Grep, WebFetch	That agent's list of tools	No command run, no file written
One agent's rule from project-management	project-manager: no raw gh changes	The hook, before each call	Refused, so it uses pkit's commands

Why pkit writes permissions down			
A rule an agent reads is advice. A rule your AI tool applies is not.	Written down, every rule can be read and reviewed, like any change.	Its own settings bind every agent alike. The hook decides per agent.	pkit's commands hold gates that raw commands skip. The agent cannot go around them.

> Limit: A deny stops the forms of a command pkit can tell. A command hidden behind another can slip past.

In project-kit: the project-manager agent is refused raw gh changes, and uses pkit's commands, which validate the issue first.

Permissions on day one

A fresh install refuses destructive commands and allows routine ones, so the prompts you still see are worth reading.

Your AI tool's settings, after install

```
# .claude/settings.json
"deny": [
  "Bash(rm -rf:*)", "Bash(sudo:*)",
  "Bash(git push --force:*)",
  "Bash(git reset --hard:*)", …
"allow": [
  "Bash(git:*)", "Bash(gh:*)", "Bash(pkit:*)",
  "Read", "Edit", "Skill(decision-author)", …
```

What pkit says about it

```
$ pkit permissions explain
all  (every agent and the operator)
  deny  destructive-fs        guardrail
  deny  privilege-escalation  guardrail
  deny  vcs-history-rewrite   guardrail
```

Refused for all

Recursive delete, sudo and force-push, for you and every agent, in every mode.

Routine, unasked

Reading, git, gh, pkit and pkit's skills run without a prompt.

Kept current

Each upgrade merges pkit's rules in again, and keeps yours.



Agents working with nobody watching?

```
$ pkit permissions setup autonomy
```

It adds a profile, the hook and the OS sandbox, except on macOS, and checks the result.

Limit: Without the hook, a rule for one agent is not applied: the settings bind every agent alike.

In a fresh install, before you write any rule: 14 forms of 4 dangerous commands refused, 37 routine ones allowed.

Part 4

Staying consistent

What pkit checks, and documents that stay true.

What pkit checks, and why

The whole project, and each change before it merges. An agent acts on what files say, and never doubts them.

The whole project

```
$ pkit validate
  manifests · schemas · configuration
  packages · connections · versions
  decisions · friction · rule-sets
  refs · process · data
  12 validator(s) ran; 0 error(s)
  All checks passed.
```

Each change, before it merges

```
$ pkit friction check
  # a document tied to what changed,
  # with no answer
$ pkit decisions check-diff
  # a record or a point numbered
  # twice, or renumbered
```

People forget

A change updates one file. What depends on it is somewhere else.

Reviews miss it

A review reads the change in front of it, not the file it left stale.

Agents trust files

An agent acts on what a file says. A check finds what it never questions.



On every push?

```
$ pkit validate && pkit friction check
```

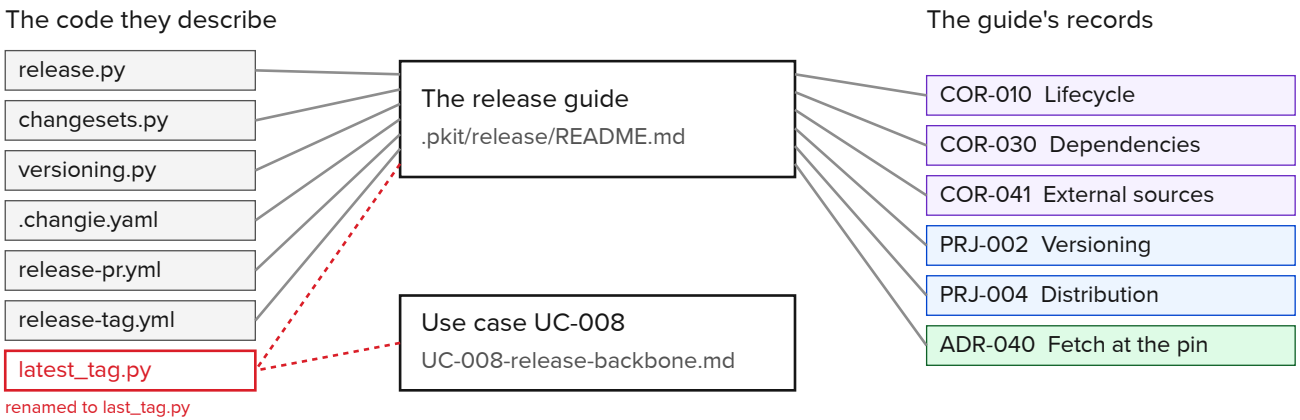
pkit adds no git hook or CI. Put both in yours, as project-kit does.

Limit: A check finds what is broken or unanswered. Whether a document is still right, a reviewer judges.

In a fresh install, pkit validate runs 12 checks with nothing to set up. Each capability brings its own: 16 in project-kit.

One tie, checked

Rename a file, and the documents tied to it are named before the change merges.




```
$ git mv scripts/latest_tag.py scripts/last_tag.py
$ pkit friction check
FINDINGS
  .pkit/release/README.md
    dead-anchor  path scripts/latest_tag.py
      matches no file; the diff removed or
      moved its target
  tech-docs/analysis/…/UC-008-release-backbone.md
    dead-anchor  path scripts/latest_tag.py
Result: failed   (2 dead anchors)
# the change fails until both ties follow the file
```

> Limit: It checks that every tie leads somewhere, not that the document is still right.

In project-kit: this check runs in the pre-push hook and on every pull request, wired by the project.

Docs that know when they're wrong

When something a document depends on changes, the change must answer for it: updated, still true, or fixed later.

The check names the document

```
# the change moves changesets to .changesets/
$ pkit friction check
  .pkit/release/README.md
    friction  path .changie.yaml  changed in this diff
              and carries no answer
Result: failed

# after the answer
$ pkit friction check
    answered  path .changie.yaml  changed in this diff;
              answered: updated
Result: passed
```

The change answers for it, in one of three ways

Updated	The change fixed the document too.
Still true	Nothing in it became wrong. The answer says why.
Fixed later	The answer names the issue that will fix it.

```
# .pkit/project/friction/answers/20261030T024530Z-ba8d160a.yaml
answers:
  - artefact: .pkit/release/README.md
    revalidated:
      outcome: updated
```

Why pkit asks for an answer

Not every change makes a document wrong. The answer says which.

A reviewer checks each answer, so "still true" needs a real reason.

A gap left open stays counted, with the issue that closes it.

> Limit: Only documents you list, with what they depend on, are covered. The rest go stale as before.

In project-kit today: 21 answers wait as "fixed later". pkit friction debt lists each, with its reason.

Part 5

What you install

Capabilities: optional parts for your kind of work, and how they connect.

Why some parts of pkit are capabilities

Core pkit holds what every project needs. The rest comes as capabilities, optional parts you choose.

Core pkit: in every project	A capability: one folder you install	
Decisions	Decision records	`decisions/`
Rules · schemas · processes	Schemas and processes	`schemas/`
Skills · agents · permissions	Skills, agents, rules	`skills/ agents/ permissions/`
Commands and checks	Commands and checks	`scripts/ package.yaml`

One command wires it, read from a fresh install

```
$ pkit capabilities install project-management
  Installed capability 'project-management' v0.54.0
  merged   .claude/settings.json      # allows its skill
  created  .claude/skills/pm/SKILL.md     # and 6 procedures
  created  .claude/agents/pm-reviewer.md
  created  .claude/agents/project-manager.md
$ pkit permissions explain
agent:project-manager
  deny  issue-tracker-write
  (contributed by capability: project-management)
$ pkit validate
  13 validator(s) ran; 0 error(s)     # 12 before
  All checks passed.
```

Why pkit has capabilities

Core holds only what every project needs. The rest would be weight.

A capability gets each upgrade, as core does. A copy would not.

One folder: its path says who owns a file.

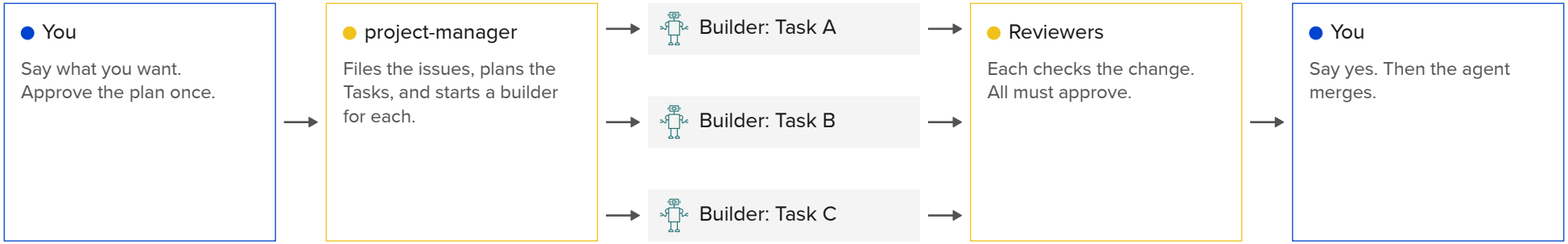
Made only for a need that kept coming back in real work.

Limit: Some capabilities need another first. software-engineering will not install without project-management.

pkit 1.149 ships 4 capabilities, and 3 more are on their way to 2.0. In a fresh install, one took a single command and 8 seconds.

What project-management adds

One agent that files the work, plans it and starts builders in parallel. You decide: you approve the plan, and each merge.



Why you hold both ends: only you commit the team to a plan or a merge. The agent does the work in between.

Its skill and agents

The pm skill, the project-manager agent, and pm-reviewer for each pull request.

88 commands

pkit pm ... files, moves and merges, and checks each step first.

Its issue workflow

Every move is checked by pkit's engine.

What you do

Give it a GitHub repository, then run pkit pm bootstrap once.

Limit: GitHub only, and large: 63 decision records and 88 commands. You meet it through the agent, not the manual.

In project-kit on 29 October: 22 pull requests merged, up to 7 open at once, each with its reviewers' verdicts.

What software-engineering adds

A builder that follows your conventions, and three reviewers of its code, on every merge that changes code.

Its agents, after install

```
$ pkit capabilities install software-engineering
  created  .claude/agents/software-engineer.md
  created  .claude/agents/code-reviewer.md
  created  .claude/agents/security-reviewer.md
  created  .claude/agents/docs-reviewer.md
# software-engineer: builds, by your conventions
# code-reviewer: is the code correct?
# security-reviewer: secrets, injection
# docs-reviewer: do the docs still hold?
```

+

Its list for the merge

```
# review-contributions.yaml
- floor: touches-code
  reviewer: code-reviewer
- floor: touches-code
  reviewer: security-reviewer
- floor: touches-code
  reviewer: docs-reviewer
- match: {type: "*"}
  reviewer: docs-reviewer
```

→

The merge, pull request #2279

```
# first round
pm-reviewer         APPROVED
docs-reviewer       APPROVED
code-reviewer       CHANGES_REQUESTED
  [block] The new Status writer
  can't find an org-owned board …
security-reviewer   APPROVED
rule-set-reviewer   APPROVED
# fixed, all five approved, merged
```

Your conventions

You write them once. The builder reads them first. They stay yours without the capability.

Code is reviewed

Before this capability, nothing in pkit reviewed code. A change with two security holes merged.

Blocks mean something

Each reviewer blocks only on a failure it can show. The rest is advice.

What you do

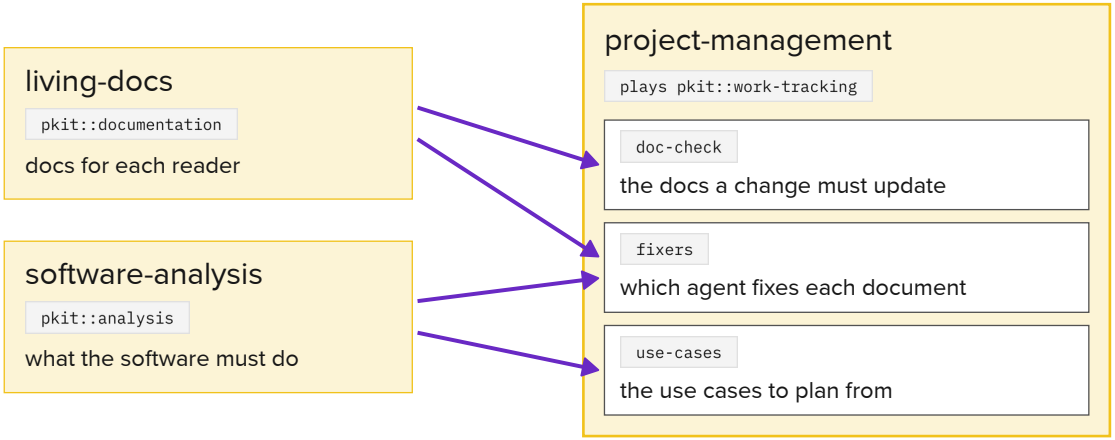
Write your conventions down, and name their file in overlay.yaml.

Limit: With no conventions written down, the builder and its reviewers fall back on general good practice.

In project-kit on 29 October: of 22 merges, the 21 that changed code each had the code, security and docs reviewers.

How capabilities work together

A capability never names another, only a role. One that needs something opens a slot, and any other can fill it.



What pkit status shows, in project-kit

```
$ pkit status
  Connections
    roles   3 named: 3 active, 0 in conflict
      pkit::analysis      → software-analysis
      pkit::documentation → living-docs
      pkit::work-tracking → project-management
    points  6 defined: 2 unfilled
    pkit::work-tracking:fixers
      data v1 · project-management — filled
      bound  software-analysis (contributes)
      bound  living-docs (contributes)
```

Why pkit connects capabilities this way

Swap a capability for one in the same role. The rest stays as it is.

Install only what you need. An empty slot does no harm.

pkit status draws the wiring, and pkit validate checks it.

> Limit: Each role has one capability so far, so swapping one for another is designed, not yet shown.

In project-kit: 3 roles, each played by one capability, and 6 slots, 4 of them filled.

Questions?

Everything shown is in project-kit's own repository, built with pkit itself.